

APPENDIX A

DATASET AND EVALUATION DETAILS

The following appendices give the dataset, training, and ablation details that complement Sec. IV of the main paper. Full rollout videos and additional qualitative comparisons are hosted on the project website¹.

A. UR5 World Model Training Dataset

We train SC3-Eval on the real-world UR5 dataset introduced in Sec. IV-A, which spans 381 hours of robot trajectories collected in a single physical scene with 12 object categories. Each trajectory provides synchronized streams from three camera views, two third-person cameras and one wrist-mounted camera, recorded at 20 Hz with native resolution 480×640 . We additionally subsample each stream to 10 Hz to form the multi-FPS training mix described in App. B-D.0.b.

The 12 object categories cover yellow cup, orange cup, blue bowl, white bowl, orange plate, chopstick, spoon, chip bag, plastic lid, plastic container, foil tray, and bottle.

Each action is recorded as a delta end-effector pose (delta-EE) and represented as a 7-dimensional vector with three components for translation, three for axis-angle rotation, and one for gripper width. We normalize each dimension to unit variance using statistics computed over the full training corpus.

B. UR5 Policy Evaluation Setup

Our real-world evaluation covers seven $\pi_{0.5}$ [26] policy checkpoints. For each checkpoint we collect 36 matched initial conditions, each rolled out both on the physical robot and inside our world-model evaluator from the same starting state for direct comparison. All rollouts, whether physical or closed-loop within the world model, run for at most 20 seconds, so that every mode shares the same temporal budget.

Tab. III reports the per-criterion real-world success rates measured in the physical-robot rollouts for all seven UR5 policy checkpoints. Each checkpoint predicts 25 actions at 10 Hz, and only the first 16 actions are executed under a receding-horizon strategy before the next replan.

TABLE III: Real-world performance of the evaluated UR5 policies. Each row reports the real-world success rate per success criterion.

Policy Checkpoint ID	Real-world success rate		
	Language following	Object lifting	Object placing
1	0.946	0.610	0.126
2	0.876	0.688	0.306
3	0.959	0.720	0.350
4	0.446	0.154	0.008
5	0.745	0.444	0.286
6	0.675	0.105	0.057
7	0.926	0.785	0.449

¹<https://miscsubmission.github.io/sc3-eval/>

C. UR5 Scoring and Annotation

This subsection details the criteria used to score each rollout and the protocol followed by human annotators.

a) *Success criteria*: Each rollout is scored on three binary criteria described in Sec. IV-A of the main paper, *language following*, *object lifting*, and *object placing*, with each criterion scored separately for every object involved in the rollout. For each rollout, three annotators score the rollout on each criterion, and the success rate for each criterion is the mean of the three annotator scores. Trajectories from the two sources (real-world and closed-loop) are rendered at a uniform 480×640 resolution and presented to annotators in randomized order. Source labels are stripped from filenames and metadata, and annotators are not told the proportion of trajectories drawn from each source. The 95% confidence intervals reported in Tab. I are estimated using 1,000 bootstrap resamples of rollouts.

b) *Failure-mode taxonomy*: For each *failed* trajectory, annotators additionally assign exactly one of the three failure modes used in Sec. IV-C of the main paper, *language-following failure*, *object-lifting failure*, or *object-placing failure*. The taxonomy mirrors the three success criteria, and each failed trajectory is assigned to the first criterion it violates (in the order language $\rightarrow$ lifting $\rightarrow$ placing).

D. YAM World Model Training Dataset

For YAM, we use the full MolmoAct2-BimanualYAM dataset [9], comprising around 720 hours of robot demonstrations, for training SC3-Eval and selecting the early-termination threshold. The held-out set used for threshold selection contains 0.5% of the dataset’s trajectories.

YAM observations comprise three camera views: left wrist, right wrist, and egocentric. Each view has an image resolution of 640×360 . We temporally downsample the data from 30 FPS to 15 FPS for training. YAM actions use the same delta end-effector pose (delta-EE) representation as UR5 (App. A-A), extended to two arms. Each arm contributes a 7-dimensional vector with three components for translation, three for axis-angle rotation, and one for gripper width, giving a 14-dimensional bimanual action vector. We use the same per-dimension unit-variance normalization procedure, with statistics computed over the YAM training corpus.

E. YAM Policy Evaluation Setup

We evaluate four policies, MolmoAct2-YAM [9], GR00T-N1.7 [27], $\pi_{0.5}$ [26], and OpenVLA-OFT [28], on object-picking tasks involving a sponge, a pen, a gray cup, and a blue cup. For MolmoAct2-YAM, we use the `allenai/MolmoAct2-BimanualYAM` checkpoint, which was fine-tuned on the bimanual YAM mixture. For GR00T-N1.7, we use the `Jiafei1224/molmoact2-yam-gr00t1.7` checkpoint; for $\pi_{0.5}$, we use `Jiafei1224/molmoact2-yam-pi05`. The GR00T-N1.7 and $\pi_{0.5}$ checkpoints were fine-tuned on the MolmoAct2-BimanualYAM dataset. For OpenVLA-OFT, we fine-tune the policy on the MolmoAct2-BimanualYAM

TABLE IV: **Real-world performance of the evaluated YAM policies.** Each entry is the mean success rate across the four object-picking tasks in the standard evaluation setting.

Policy	Real-world success rate	
	Language following	Object lifting
MolmoAct2-YAM	0.425	0.250
GR00T-N1.7	0.738	0.438
$\pi_{0.5}$	0.675	0.400
OpenVLA-OFT	0.325	0.150

dataset using the official [moojink/openvla-oft](https://github.com/moojink/openvla-oft) implementation with 20k training steps.

Tab. IV summarizes the real-world success rates of the four evaluated YAM policies, averaged across the four object-picking tasks.

As in the UR5 setup, real-world rollouts and closed-loop rollouts generated by the world model start from matched initial conditions, following Sec. IV-A. Each policy is evaluated on all four objects, with 20 trajectories per object, for a total of 80 trajectories per policy. Both real-world and generated YAM rollouts run for at most 8 seconds.

a) *Environmental distribution shifts*: The shift experiments are limited to the policy-task combinations reported in Fig. 8. For the object-layout shift, we move the pen from the gray cup to the blue cup and evaluate blue-cup lifting with OpenVLA-OFT and MolmoAct2-YAM. For the object-color shift, we replace the blue cup with a yellow cup and evaluate language following on the sponge-picking task with MolmoAct2-YAM and GR00T-N1.7. For each shift, we pair the original and modified conditions by initial scene configuration and hold all other conditions fixed. For both shifts, the language prompt for each task remains unchanged between the original and modified conditions. For each evaluated policy, we use 20 trajectories per object in each of the original and modified conditions.

F. YAM Scoring and Annotation

YAM follows the same blinded human-annotation and success-rate aggregation protocol as UR5 (App. A-C), but uses only two binary success criteria: *language following* and *object lifting*. Confidence intervals use the same rollout-level bootstrap procedure.

APPENDIX B TRAINING DETAILS

This appendix describes the training and inference configurations for the UR5 and YAM experiments.

A. Network Architecture

To leverage prior knowledge from vision foundation models, we initialize the network from the pre-trained weights of Cosmos3-Nano, the smallest variant of the Cosmos3 family [11].

Raw delta-EE actions are first normalized per-dimension as described in App. A-A, then projected through an MLP

into action embeddings. The resulting embeddings are injected as additive tokens into the time-embedding pathway of each Diffusion Transformer block, and the same action-conditioning pathway is shared between the forward and inverse dynamics objectives.

B. Hardware and Compute

We train SC3-Eval on 32 NVIDIA GB200 GPUs (8 nodes with 4 GPUs per node) for approximately 2.2 wall-clock days. Inference for closed-loop evaluation runs at 2.3 seconds per 24-frame chunk on a single GB200 GPU.

C. Optimization

a) *Optimizer and schedule*: We use the AdamW optimizer with learning rate 0.0001, weight decay 0.05, $\beta_1 = 0.9$, $\beta_2 = 0.99$, and a 2,000-step linear warmup followed by cosine decay. The effective batch size is 512 trajectories per step (16 per-GPU minibatch across 8 nodes with 4 GB200 GPUs per node), and we train for 24,000 total steps with the rectified-flow loss inherited from Cosmos3.

b) *Mode mixture*: Following the training procedure in Sec. III-B, we use the following mode-sampling probabilities. For UR5, we set $p_{\text{FD}} = 0.8$ for multi-view forward dynamics, $p_{\text{CVI}} = 0.1$ for cross-view inpainting, and $p_{\text{ID}} = 0.1$ for inverse dynamics. For YAM, we set $p_{\text{FD}} = 0.9$, $p_{\text{CVI}} = 0.05$, and $p_{\text{ID}} = 0.05$.

D. Training Augmentations

a) *Pseudo-action augmentation*: With probability $p_{\text{pseudo}} = 0.5$, the policy command for a minibatch sample is replaced by a pseudo-action computed from consecutive real-world end-effector poses. The pseudo-action and policy-command distributions occupy slightly different action manifolds, and exposing the model to both during training provides marginal robustness gains.

b) *Multi-FPS training*: For UR5, we jointly train at 10 and 20 FPS by sampling the temporal stride uniformly per minibatch. The two FPS settings correspond to effective action horizons of 2.4 and 1.2 seconds for the 24-step chunks used during training. They expose the model to complementary temporal statistics within a fixed chunk budget. The 20 FPS stream provides fine-grained inter-frame information that is useful for short-range physics and contact, while the 10 FPS stream covers twice the temporal span and exposes the model to more global movement per chunk. Since the transformer does not directly observe the frame rate through its positional encoding, this is best read as a data-augmentation effect that broadens the visual statistics each chunk represents, rather than as an explicit timescale-conditioning signal. For YAM, we train at 15 FPS only, which corresponds to an effective action horizon of 1.07 seconds for the 16-step chunks used during training.

E. Inference Procedure

a) *Prediction-execution horizon decoupling*: Following the notation of Sec. III-C of the main paper, we decouple the world model’s prediction horizon from the policy’s execution

TABLE V: **Open-loop rollout PSNR per method, ablation variant, and camera view.** Results are reported separately for the three camera views of UR5 and YAM. PSNR is averaged over the first 50 generated frames of each rollout. Higher is better.

Method	UR5 PSNR $\uparrow$			YAM PSNR $\uparrow$		
	Third-person 1	Third-person 2	Wrist	Left wrist	Right wrist	Egocentric
Ctrl-World [6]	19.90	19.92	19.01	19.86	19.79	20.51
IRASim [3]	19.44	19.47	18.60	19.01	19.04	20.00
Cosmos-Predict 2.5 [10]	19.86	19.88	19.07	19.82	19.80	20.04
Cosmos3-FD [11]	20.41	20.73	19.92	20.37	20.45	21.19
w/o inverse dynamics	21.03	21.05	19.43	19.93	19.91	21.56
w/o cross-view inpainting	21.33	21.35	19.71	19.71	19.69	21.19
w/o horizon decoupling	20.96	20.98	19.37	19.99	19.97	21.89
SC3-Eval (Ours)	21.46	21.49	20.64	21.39	21.37	22.10

horizon. For UR5, the world model is trained at a prediction horizon of $l' = 24$ frames, of which only the first $l = 16$ frames are retained at inference, matching the policy’s 16-step receding-horizon execution. Training directly at $l = 16$ frames degrades generation quality, since this horizon is far shorter than the 400-frame video clips on which Cosmos3 was pre-trained. At each closed-loop step the policy proposes a 25-action chunk, of which we feed the first $l' = 24$ actions to the world model. The world model generates l' future frames in each of the three camera views, and we keep the first l frames per view as the observation context for the next chunk. For YAM, SC3-Eval uses a world-model prediction horizon of $l' = 16$ frames and an execution horizon of $l = 8$ steps.

b) Early-termination threshold: For the uncertainty-driven early-termination criterion used in Sec. IV-E, we tune the UR5 per-chunk threshold to $\tau = 0.02$ on a held-out subset of 30 trajectories. The UR5 threshold-sensitivity analysis is provided in App. C-C. For YAM, we enable uncertainty-driven early termination with threshold $\tau = 0.45$, selected using the held-out subset described in App. A-D. Following the main-paper protocol, terminated rollouts remain included in the success-rate computation, and annotators score each criterion using only the retained trajectory prefix.

APPENDIX C ADDITIONAL ABLATION STUDY

The main paper evaluates each design choice via closed-loop policy-level correlation metrics in Tab. III. This appendix complements that analysis by examining how each design factor affects the visual quality of open-loop generation. Tab. V reports rollout PSNR against the real-world frames by method, ablation variant, and camera view for UR5 and YAM. PSNR is averaged over the first 50 generated frames of each rollout. Closed-loop rollouts depart from the real-world frames by construction, so PSNR is meaningful only in open-loop mode. Subsequent subsections analyze each design factor in turn, and App. C-C additionally examines how the early-termination threshold affects closed-loop policy evaluation across all seven UR5 checkpoints.

A. Training Modes

SC3-Eval introduces two grounding training modes alongside multi-view forward dynamics (Sec. III-B), inverse dynamics and cross-view inpainting. We ablate each in turn below.

a) Inverse dynamics: Removing the inverse dynamics grounding objective reduces open-loop PSNR in every camera view on both platforms (Tab. V, w/o inverse dynamics row). The corrective signal from inverse dynamics becomes most valuable when policy commands drive the rollout off the training-data manifold, where the rollout otherwise drifts without correction. Fig. 9 illustrates this qualitatively on a single open-loop trajectory, where the variant without inverse dynamics tracks the ground truth for the first second and then diverges onto a different scene, while the variant with inverse dynamics follows the bottle pick-and-place motion through the end of the rollout.

b) Cross-view inpainting: The cross-view inpainting ablation has a particularly striking qualitative effect on wrist-view scene re-entry. When the wrist camera temporarily turns away from the workspace and later returns, the variant trained without cross-view inpainting hallucinates the workspace and never recovers once it becomes visible again, while the variant trained with cross-view inpainting restores the correct workspace state. We identify re-entry segments automatically as rollout windows in which the wrist camera workspace visibility transitions from below threshold to above threshold. Fig. 10 compares both variants over the same re-entry segment at four timesteps, with the zoomed wrist view on top and the two third-person views below for each panel.

B. Prediction-Execution Horizon Decoupling

The prediction-execution horizon decoupling result in Tab. III of the main paper carries over to the open-loop mode, with the w/o horizon decoupling row of Tab. V showing a PSNR drop in every camera view on both platforms. Setting $l' = l$ (no decoupling) shortens the per-chunk video horizon below the world model’s pre-training prior, and degrades both closed-loop policy-level correlation and open-loop visual quality.

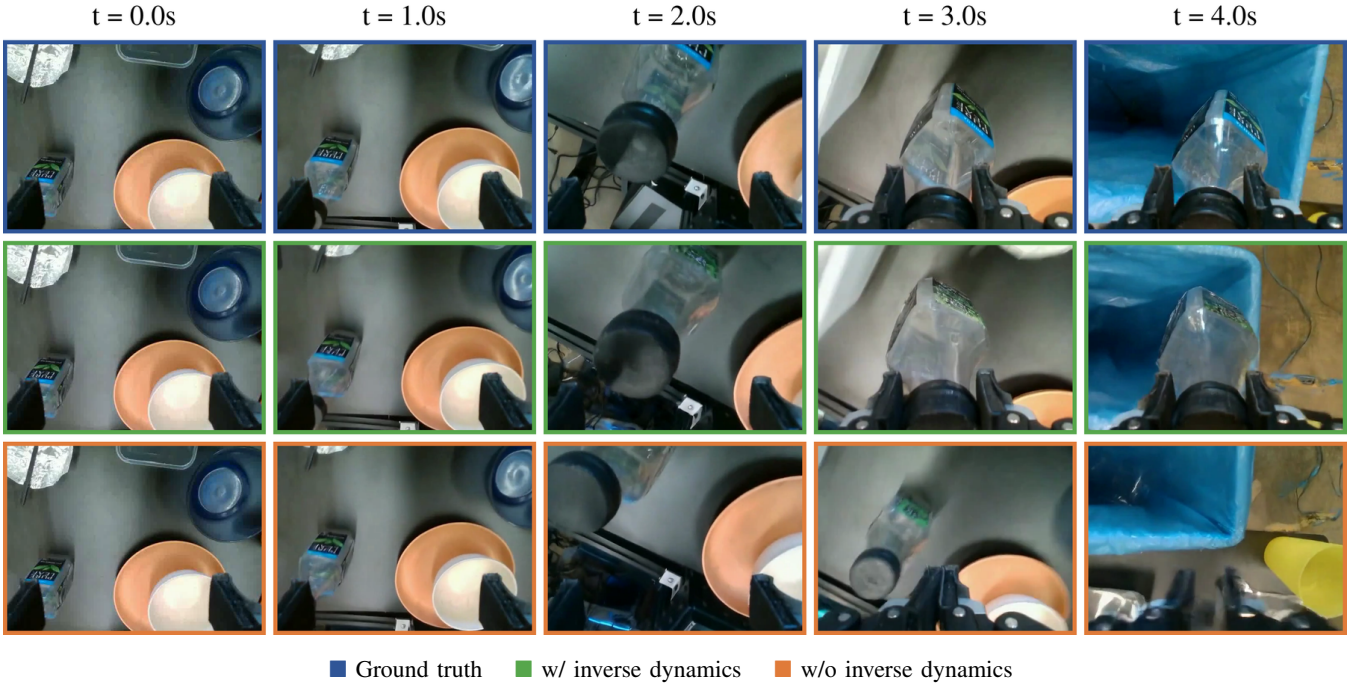


Fig. 9: **Qualitative effect of inverse dynamics grounding on open-loop rollouts.** Each row shows the same open-loop rollout under a different training configuration, sampled at five timesteps from a single bussing trajectory. The variant trained without inverse dynamics matches the ground truth for the first second, then drifts onto a different scene and never recovers, while the variant with inverse dynamics tracks the ground-truth bottle pick-and-place motion through the end of the rollout.

TABLE VI: **Uncertainty-driven early termination ablation on UR5.** Pearson r and MMRV are computed separately for each success criterion across all seven checkpoints, then averaged over the three criteria. Bold indicates the best result among the evaluated settings.

Threshold τ	Termination rate	Mean Pearson $r \uparrow$	Mean MMRV $\downarrow$
∞ (no termination)	0%	0.894	0.080
$\tau_1 = 0.05$	2.21%	0.899	0.075
$\tau_2 = 0.02$	4.88%	0.948	0.036
$\tau_3 = 0.01$	10.22%	0.901	0.128

$\tau_2 = 0.02$ to $\tau_3 = 0.01$ raises the termination rate to 10.22%, but lowers mean Pearson and increases mean MMRV. At τ_3 , mean MMRV is worse than the no-termination baseline, although mean Pearson remains slightly higher. Thus, more frequent termination does not necessarily improve evaluation accuracy and can impair policy ranking.

C. Uncertainty-Driven Early Termination

Tab. VI reports the uncertainty-driven early-termination ablation across all seven UR5 policy checkpoints, together with the termination rate (fraction of rollouts terminated by the criterion) at each threshold. Pearson r and MMRV are computed separately for each success criterion and then averaged over the three criteria, following the main-paper protocol. At the adopted threshold $\tau_2 = 0.02$, early termination improves mean Pearson r from 0.894 to 0.948 and reduces mean MMRV from 0.080 to 0.036 relative to no termination, while terminating 4.88% of rollouts. This setting achieves the highest mean Pearson and lowest mean MMRV among the evaluated settings, supporting its use for policy evaluation. The less aggressive threshold $\tau_1 = 0.05$ terminates 2.21% of rollouts and yields only small improvements over no termination. Tightening the threshold from

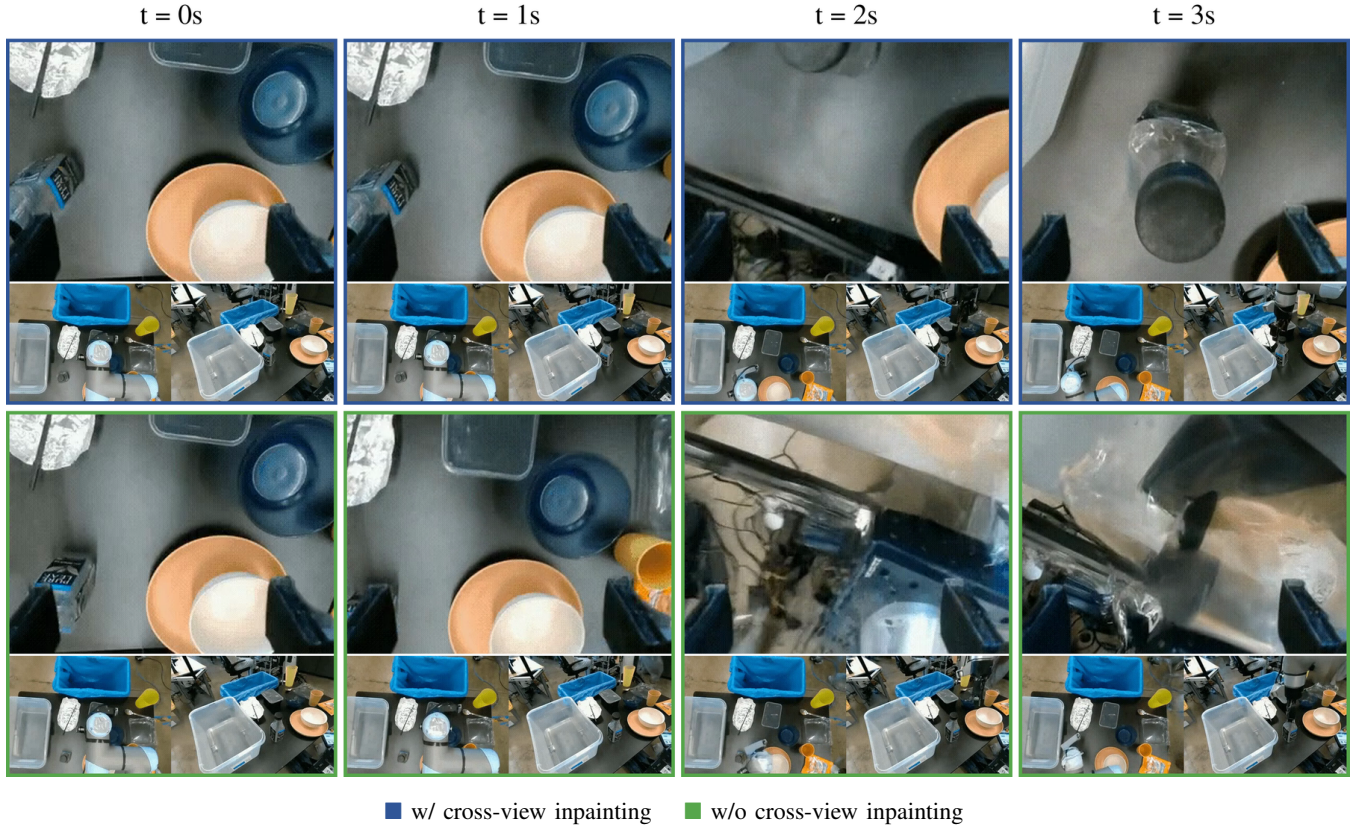


Fig. 10: **Qualitative effect of cross-view inpainting on wrist-view scene re-entry.** Each row shows the same open-loop rollout under a different training configuration, sampled at four timesteps spanning a re-entry segment in which the wrist camera turns away from the workspace and then returns. Each panel shows a zoomed wrist view on top and the two third-person views below. The variant trained without cross-view inpainting hallucinates the wrist content and never resolves once the workspace becomes visible again, while the variant trained with cross-view inpainting recovers the correct workspace state under occlusion.

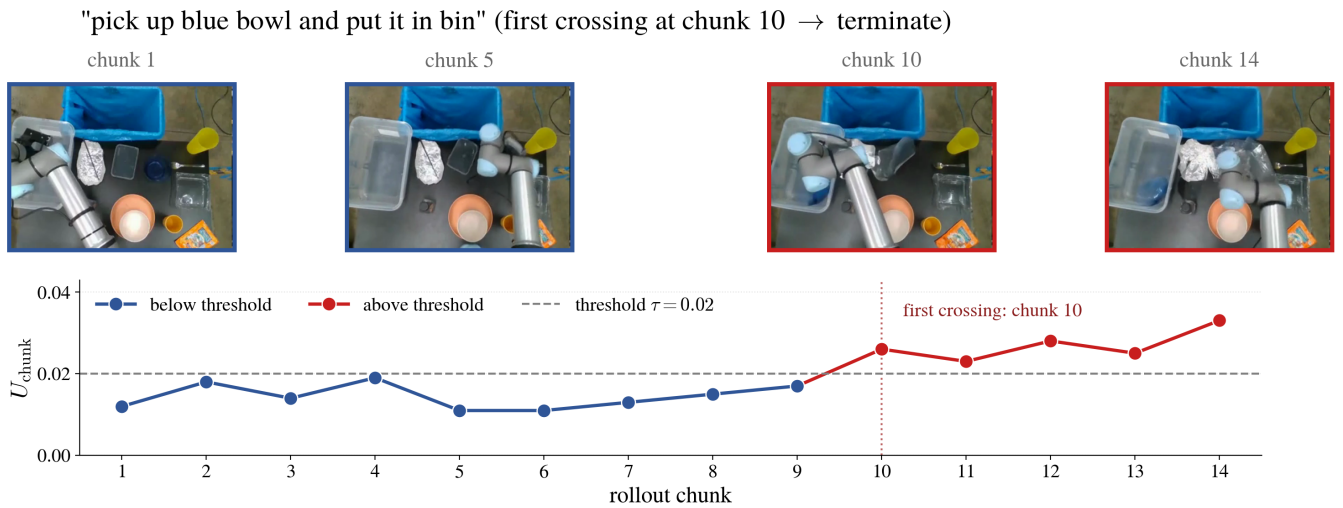


Fig. 11: **Uncertainty-driven early termination on an example rollout.** Top, four frames sampled from the imagined rollout. Bottom, the per-chunk consistency error U_{chunk} over the 14 rollout chunks. The rollout is terminated at the first chunk whose U_{chunk} exceeds the threshold $\tau = 0.02$ (chunk 10 here), before the visible drift compounds.